Random Self Reducibility

This report was written as part of the 2025 edition of the self-study PhD course "Cryptographic Proof Techniques" by Prof. Elena Pagnin at Chalmers University of Technology, Göteborg, SE.

1 Random-self-reducible functions

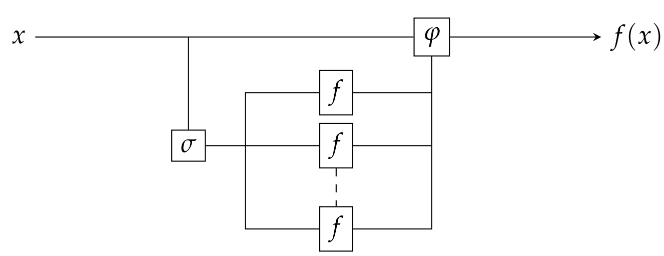
Without loss of generality, we can define a *decision problem* as a set D of binary strings where we are asked if a given $x \in \{0,1\}^*$ lies in D. Similarly, a *search problem* is a set S of binary strings (x, y) where given x we are asked to find an y such that $(x, y) \in \mathsf{S}$.

▶ **Definition 1** *A* poly-time reduction from problem P_1 to problem P_2 *($\mathsf{P}_1 \leq_p \mathsf{P}_2$) is a poly-time algorithm transforming any instance x_1 of P_1 into an instance x_2 of P_2 such that $x_1 \in \mathsf{P}_1$ iff $x_2 \in \mathsf{P}_2$.*

Consider a function $f : \{0,1\}^* \to \{0,1\}^*$, taking inputs x of $\mathsf{len}(x) = n$. We denote by r a sequence of fair coin tosses, with $\mathsf{len}(r) = \omega(n)$ for $\omega(n) = \mathsf{poly}(n)$.
We wish to study reductions running in probabilistic polynomial time. They will perform $k(n) = \mathsf{poly}(n)$ random queries to f. We consider the following definitions, found for instance in Feigenbaum and Fortnow [1].

▶ **Definition 2** *A function $f : \{0,1\}^* \to \{0,1\}^*$ is* nonadaptively $k(n)$-random-self-reducible *(nonadaptively k-rsr) if there are two polynomial-time computable functions σ and φ such that*

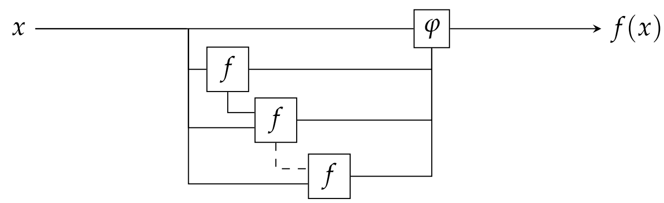
1. *for all n and all $x \in \{0,1\}^n$, for at least $3/4$ of all r in $\{0,1\}^{\omega(n)}$, we can compute $f(x)$ as*
$$f(x) = \varphi\big(x, r, f(\sigma(1, x, r)), \ldots, f(\sigma(k, x, r))\big)$$
2. *for all n, if r is chosen uniformly at random, then the random variables $\sigma(i, x_1, r)$ and $\sigma(i, x_2, r)$ are identically distributed for all $\{x_1, x_2\} \in \{0,1\}^n$ and all $i = 1, \ldots, k$.*



Intuitively, f is nonadaptively k-rsr if evaluating f on any size-n instance x can be reduced via φ to evaluating f on k instances $\sigma(1, x, r), \ldots, \sigma(k, x, r)$ randomised by the same uniformly random variable r modelling $\omega(n)$ fair coin tosses. Here n can be treated as a security parameter.
Feigenbaum and Fortnow generalise this approach to multiround adaptive strategies.

▶ **Definition 3** *A function $f : \{0,1\}^* \to \{0,1\}^*$ is* adaptively $k(n)$-random-self-reducible *(adaptively k-rsr) if there is a poly-time oracle machine φ that, on input x of $\mathsf{len}(x) = n$, produces $k(n)$ rounds of oracle f-queries $y_i(x,r)$ to f, where the i-th query might depend on previous queries and answers. The reduction φ must be such that*

1. *for all n and all $x \in \{0,1\}^n$, for at least $3/4$ of all r in $\{0,1\}^{\omega(n)}$, it outputs the correct answer $f(x)$.*
2. *for all n, if r is chosen uniformly at random, then the random variables $y_i(x_1,r)$ and $y_i(x_2,r)$ are identically distributed for all $\{x_1,x_2\} \in \{0,1\}^n$ and all $i = 1,\dots,k$, except if wrong answers were given on earlier rounds.*



Intuitively, the key difference lies in the strategy for generating the queries. Nonadaptive k-rsr requires the reduction φ to generate all oracle queries $\sigma(i,x,r)$ in parallel, solely based on the input x and the initial randomness r. Adaptive k-rsr allows it to proceed sequentially over k rounds, where the i-th query y_i may depend on $y_1,\dots,y_{i-1}$. This latter approach is for instance better suited when the solution is found narrowing down the solution space.

Clearly, if an adaptively $k(n)$-rsr function f is also nonadaptively $k(n)$-rsr, but the converse was shown to be false by Feigenbaum et al. [2] with the construction of a set L'' such that $f = \chi_{L''}$ is adaptively k-rsr. For the sake of convenience, a function f is called *poly-rsr* if it is adaptively or non-adaptively k-rsr for a polynomial number of queries $k = k(n)$.
We are interested in the case where f is a function solving the computational problem behind a cryptographic security assumption, which are often nonadaptive 1-rsr functions.

2 Rsr problems for classical cryptography

2.1 The Discrete Logarithm problem

▶ **Problem 4 (DLOG)** *Consider a finite cyclic group $\mathbb{G} = \langle g \rangle$ of order n.*

- *Parameters: a description $\Gamma = (\mathbb{G}, g)$ of the group $\mathbb{G}$.*
- *Instance: an element $h \in \mathbb{G}$.*
- *Task: find the unique $e \in \mathbb{Z}_n$ such that $g^e = h$.*

For a nice analysis of DLOG, see Joux et al [3]. The following lemma is often proved without most of the formalism in Definition 2. However, we will maintain it for didactic purposes.

▶ **Lemma 5** *The $DLOG_{\mathbb{G},g}$ problem is poly-rsr.*

Proof. Given a fixed instance $x = h \in \mathbb{G}$, the discrete logarithm problem requires us to find $f(x) = \log_g(x)$. Sample an uniformly random integer $r \in \mathbb{Z}_n$ of length $\omega(\mathsf{len}(x)) = \mathsf{len}(x)$, which is $\mathsf{poly}(\mathsf{len}(x))$, then $\sigma(1, x, r) = hg^r$ behaves as an uniformly random variable in $\mathbb{G}$. Given two instances $x = g^e$ and $\tilde{x} = g^{\tilde{e}}$, consider $y = \sigma(1, x, r)$ and $\tilde{y} = \sigma(1, \tilde{x}, r)$, then for some arbitrary $z = g^a \in G$ and a truly uniformly random variable r

$$\begin{aligned}
\mathbb{P}[y = z] &= \mathbb{P}[xg^r = z] = \mathbb{P}[g^{e+r} = g^a] \\
&= \mathbb{P}[r = a - e] = \frac{1}{n} \\
&= \mathbb{P}[r = a - \tilde{e}] = \mathbb{P}[\tilde{x}g^r = z] \\
&= \mathbb{P}[\tilde{y} = z]
\end{aligned}$$

Finally, we exhibit the reduction $\varphi(x, r, f(\sigma(1, x, r))) = f(\sigma(1, x, r)) - r$, which is correct since

$$\begin{aligned}
\varphi(x, r, f(\sigma(1, x, r))) &= \varphi(h, r, f(hg^r)) \\
&= f(hg^r) - r \\
&= e + r - r = f(x)
\end{aligned}$$

This function satisfies the condition of 2, thus the problem is nonadaptive k-rsr for $k = 1$. $\square$

Remark how we exploited the omogeneous properties of f, namely $f(xg^r) = f(x) + r$.

2.2 The RSA Inversion problem

▶ **Problem 6 (RSA Inversion)** *Consider an RSA modulus $N = pq$ with distinct primes p, q, and a public exponent e such that $\gcd(e, \varphi(N)) = 1$.*

- *Parameters: the system description (N, e).*
- *Instance: an element $x \in \mathbb{Z}_N^*$.*
- *Task: find the unique $y \in \mathbb{Z}_N^*$ such that $y^e \equiv x \mod N$*

▶ **Lemma 7** *The $RSA_{N,e}$ problem is poly-rsr.*

Proof. Given a fixed instance $x \in \mathbb{Z}_N^*$, the RSA inversion problem requires us to find $f(x) = y$, the e-th root of x modulo N. Sample an uniformly random integer $r \in \mathbb{Z}_N$ of length $\omega(\mathsf{len}(x)) = \mathsf{len}(x)$, which is $\mathsf{poly}(\mathsf{len}(x))$. Define $\sigma(1, x, r) = xr^e$, this behaves as an uniformly random variable in $\mathbb{Z}_N$. Given two instances $x = y^e$ and $\tilde{x} = \tilde{y}^e$, define $z = \sigma(1, x, r)$ and $\tilde{z} = \sigma(1, \tilde{x}, r)$, then for some arbitrary instance $\overline{x} = \overline{y}^e \in \mathbb{Z}_N^*$ and a truly uniformly random variable r

$$\begin{aligned}
\mathbb{P}[z = \overline{x}] &= \mathbb{P}[xr^e = \overline{y}^e] = \mathbb{P}[(yr)^e = \overline{y}^e] \\
&= \mathbb{P}[r = \overline{y}/y] = \frac{1}{\varphi(N)} \\
&= \mathbb{P}[r = \overline{y}/\tilde{y}] = \mathbb{P}[\tilde{x}r^e = \overline{y}^e] \\
&= \mathbb{P}[\tilde{z} = \overline{x}]
\end{aligned}$$

We define the reduction $\varphi(x, r, f(\sigma(1, x, r))) = f(\sigma(1, x, r))/r$, which is correct since

$$\begin{aligned}\varphi(x, r, f(\sigma(1, x, r))) &= \varphi(x, r, f(xr^e)) \\ &= yr/r = f(x)\end{aligned}$$

This function satisfies the condition of 2, thus the problem is nonadaptive k-rsr for $k = 1$. $\square$

3 Rsr problems for post-quantum cryptography

3.1 The Endomorphism Ring problem

An *elliptic curve* is an algebraic variety

$$E = \left\{ (x, y) \in \bar{\mathbb{F}}^2 \cup \{\mathcal{O}\} \mid y^2 = x^3 + Ax + B \right\}$$

where the discriminant $\Delta = -16(4A^3 + 27B^2)$ is non-zero. For theoretical reasons[1] we focus on elliptic curves over $\mathbb{F}_{q^2}$. An *isogeny* is a surjective morphism $\phi : E \to E'$ such that for amy $P, Q \in E$ we have $\phi(P + Q) = \phi(P) + \phi(Q)$. Its *degree* is the degree of φ as a rational function, and its *representation* is any data that encodes the domain, the codomain, the degree, and an algorithm to evaluate the image of any point.

The book by Silverman [4] provides a more rigorous introduction to elliptic curves. For the applications to cryptography and efficient representations, see the survey by Robert [5].

► **Problem 8 (Endomorphism Ring)** *Consider the finite field $\mathbb{F}_{q^2}$.*

- *Parameters: the field size q^2.*
- *Instance: a supersingular elliptic curve E over $\mathbb{F}_{q^2}$.*
- *Task: find the endomorphisms generating* $\mathrm{End}(E)$ *as a four-dimensional $\mathbb{Z}$-module.*

We will only provide a sketch of proof for the following Lemma, which would require an heavy introduction to the theory of elliptic curves, isogenies, and efficient representations. Still, it is worth exploring the fact that the proof requires relaxing the definition of nonadaptive k-rsr function. It is unknown whether it can be reduced in poly-time to Definition 2.

► **Lemma 9** *If we allow for the statistical indistinguishability of the distribution of the random variables $\sigma(i, x, r)$ in Definition 2 , EndRing$_q$ is poly-rsr.*

Sketch of proof. Consider a fixed instance $x = E$ of OneEnd_{q^2}. The random variables $\sigma(i, x, r)$ sample a random isogeny ψ_i with domain in E as follows:

- Via Velu's formulae [4], we can uniquely characterise any isogeny ψ_i via its kernel $\ker(\psi_i)$. The degree $\deg(\psi_i)$ is the order of the points generating $\ker(\psi_i)$.

[1] $\mathbb{F}_{q^2}$ is the smallest field extension of $\mathbb{F}_q$ where all the 2-torsion points (points P such that $2P = \mathcal{O}$) are defined.

- Construct a degree-2^ℓ isogeny ψ_i, where $2^\ell > q^2$. This is done via a computational trick, which is constructing a degree-2 isogeny ψ_2 and then composing it ℓ times:

$$\psi_i = \underbrace{\psi_2 \circ \psi_2 \circ \cdots \circ \psi_2}_{\ell \text{ times}}$$

- The isogeny ψ_2, in turn, is constructed by sampling a basis P_1, P_2 of the 2-torsion subgroup $E[2]$, the set of points of order 2 in E.

We need to sample isogenies of degree 2^ℓ,where ℓ is actually $\ell > \log_2 p^2 + \epsilon$. According to the Mixing Time Theorem [6, Theorem 11], doing this for an adequate ε guarantees a stationary distribution in the supersingular isogeny graph. Define the elliptic curve $E'_i = \sigma(i, x, r) = \psi_i(E)$. The Mixing Time Theorem claims it comes from a distribution $\mathcal{D}_\ell$ on the supersingular isogeny graph that is statistically indistinguishable from the uniform distribution.
We have that E'_i is a new randomised instance of EndRing_q. The reduction φ, on input the four generators $\iota'_1, \iota'_2, \iota'_3, \iota'_4$ of $End(E'_i)$, returns

$$\iota_i = \psi_i \circ \iota'_i \circ \hat{\psi}_i$$

$$\begin{tikzcd} E \arrow[r, "\psi_i"] \arrow[d, "\iota"'] & E'_i \arrow[d, "\iota'_i"] \\ E & E'_i \arrow[l, "\hat{\psi}_i"] \end{tikzcd}$$

where $\hat{\psi}_i$ is the dual isogeny of ψ_i, computable in poly-time.
This procedure has some failure rate, as we could end in a subring $R \subsetneq \text{End}(E)$, but it can be proved to satisfy the bound in Definition 2 for $k = 1$. ☐

3.2 The Group Action Inverse problem

Consider a multiplicative group G and a set X. Many of the post-quantum security assumptions in NIST standards and competitors can be modeled with the formalism of group actions.

▶ **Definition 10** *We say that G acts on X if there exists a map $\star : G \times X \to X$ such that*

- *for every $x \in X$, we have $e \star x = x$, where e is the identity element*
- *for every $g, h \in G$ and every $x \in X$, we have $(gh) \star x = g \star (h \star x)$*

This map $\star$ is called group action*.*

A group action partitions X into *orbits*; the orbit associated to x is the set $G_x = \{g \star x \mid g \in G\}$. Therefore a group action is transitive if and only if it only has one orbit.
For a group action to be suitable for cryptographic use, we require X, G to be finite and the existence of probabilistic poly-time algorithms performing the following operations:

1. **Membership.** Given a string g, decide if it represents an element of G. Given a string x, decide if it represents an element of X.
2. **Equality.** Given $g, h \in G$, decide if $g = h$.
3. **Random sampling.** Sample an element in G with given distribution $\mathcal{D}_G$ on G.
4. **Group operations.** Given $g, h \in G$, compute g^{-1} and gh.
5. **Action.** Given $g \in G$ and $h \in H$, compute $g \star h$.
6. **Representation.** Given an element $x \in X$ or $x \in X$, return the (not necessarily unique) representation as string in $\{0,1\}^{\#G}$.

Denote by $\delta(y, x)$ the element(s) g mapping to y from x, i.e. $y = g \star x$. If the action is transitive, g is unique.

▶ **Problem 11 (Group Action Inverse)** *Consider a set X and a group G.*

- *Parameters: a group action $\star$ of G on X.*
- *Instance: two elements x, y of the set X.*
- *Task: find, if any, an element g such that $x = g \star y$.*

If the group action is transitive, this g must exist. We will prove this problem is poly-rsr, a result first formalised by D'Alconzo [7].

▶ **Lemma 12 ([7])** *If $\star$ is transitive, $GAIP_{G,X,\star}$ is poly-rsr.*

Proof. Consider the fixed instance (x, y) of GAIP. we are asked to find an $f(x, y)$ such that $x = f(x, y) \star y$. The function $\sigma(1, x, r)$ generates two uniformly random group elements $g_{1,x}$ and $g_{1,y}$, returning

$$\sigma(1,(x,y),r) = (g_{1,x} \star x, g_{1,y} \star y) = (x', y')$$

The outputs of σ are truly uniformly distributed: consider the random variables

$$\begin{aligned}(x', y') &= \sigma(1,(x,y),r)\\ (\tilde{x}', \tilde{y}') &= \sigma(1,(\tilde{x},\tilde{y}),r)\end{aligned}$$

and two $z, w \in X$. If σ samples truly uniformly random group elements, then

$$\begin{aligned}\mathbb{P}[(x', y') = (z, w)] &= \mathbb{P}[(g_{1,x} \star x, g_{1,y} \star y) = (z, w)]\\ &= \mathbb{P}[g_{1,x} = \delta(x, z), g_{1,y} = \delta(y, w)]\\ &= \frac{1}{|G|^2} = \mathbb{P}[g_{1,\tilde{x}} = \delta(\tilde{x}, z), g_{1,\tilde{y}} = \delta(\tilde{y}, w)]\\ &= \mathbb{P}[(\tilde{x}', \tilde{y}') = (z, w)]\end{aligned}$$

Finally, we define the function φ as

$$\begin{aligned}\varphi\big((x,y), r, f(\sigma(1,(x,y),r))\big) &= \varphi\big((x,y), r, f(x', y')\big)\\ &= g_{1,x}^{-1} \cdot f(x', y') \cdot g_{1,y}\end{aligned}$$

$$
\begin{tikzcd}
x \arrow[r, "g_{1,x}"] \arrow[d, "f(x,y)"'] & x' \arrow[d, "f(x',y')"] \\
y \arrow[r, "g_{1,y}"'] & y'
\end{tikzcd}
$$

From the commutative diagram we can see that the reduction φ is correct with probability 1. □

An example of group action is given by Linear Code Equivalence, the computational problem behind the code-based signature scheme LESS [8].

▶ **Problem 13 (Linear Code Equivalence)** *Consider linear codes defined over $\mathbb{F}_q$.*

- *Parameters: code parameters n, k, q.*
- *Instance: the generator matrices G, G' of two (n, k)-codes C, C'.*
- *Task: find, if any, $S \in GL_k(\mathbb{F}_q)$ and $Q \in M_n(\mathbb{F}_q)$ such that $G' = SGQ$.*

This is an instance of Group Action Inversion, where the action

$$
\begin{aligned}
\star : G \times X &\longrightarrow X \\
((S; (\alpha, Q)), M) &\longmapsto S \cdot \alpha(MQ)
\end{aligned}
$$

is induced on the set of full-rank $k \times n$ matrices by the group $G = GL_k(\mathbb{F}_q) \times (\mathrm{Aut}(\mathbb{F}_q) \rtimes M_n(\mathbb{F}_q))$, whose elements are a triple $g = (S; \alpha, Q)$ of an invertible matrix S, a field automorphism α, and a monomial matrix Q. The operation $g \star M$ mixes and scales the columns of M (multiplication by Q), applies an automorphism (α) and performs a change of basis (multiplication by S).
However, **LCE is not poly-rsr**. Codes with different invariants, like weight distribution, cannot be mapped one into the other.

4 Consequences

4.1 One-way functions

Angluin and Lichtenstein in [9] remark how most classical security assumptions are based on random self-reducible problems: quadratic residuals, RSA inversion, discrete logarithm. Some we saw in Section 2, and post-quantum ones in Section 3. A suggested reason is that the class of poly-rsr functions seemingly gives rise to one-way functions.

Abadi et al. [10] consider *instance hiding schemes*, where a secret instance x is efficiently transformed into a uniformly random instance y such that $f(y)$ allows recovery of $f(x)$. This allows users to rely on possibly malicious programs.
For example, in the Discrete Logarithm Problem, finding x in $h = g^x$ can be transformed by choosing a random r and querying the solution for $h' = h \cdot g^r = g^{x+r}$. The resulting solution x' then yields $x = x' - r$. This mechanism enables verifiable yet private computation of a compatible γ,

as the program only solves a random-looking problem instance, preserving the confidentiality of the original secret x.

▶ **Corollary 14** *Since DLOG is nonadaptively* 1*-rsr, then it is a* 1*-ihs (instance hiding scheme), meaning that an instance of DLOG can be hidden by one other instance.*

This can be generalised to nonadaptively $k(n)$-rsr functions using the same φ and σ of Definition 2.

Further, random-self-reducibility allows reliable use of a flawed program P to evaluate f. This is similar to an instance hiding scheme, but we are unsure on the correctness of the output of P.
Consider a program P evaluating f that is correct on at least $1/2$ of inputs. Compute i randomised instances $y_i = \sigma(i, x, r)$ from the correct input x. Once the flawed output is obtained, invert the reduction with φ. By repeating this process multiple times, the correct answer $f(x)$ is likely the majority result, effectively mitigating the error rate of P. As above, this application makes clever use of the functions in Definition 2, which introduce a noticeable overhead. Blum et al. [11] define this property *self-correction*.

4.2 Weak instances

A cryptographic protocol is secure only if the underlying mathematical problem is hard to solve for the vast majority of instances, but average-case hardness is hard to prove for general problems. RSR provides the necessary guarantee that worst-case hardness is sufficient.

▶ **Corollary 15** *Consider a poly-rsr function f. If an algorithm $\mathcal{A}$ correctly solves $f(x)$ for a non-negligible fraction $\rho(n) > 0$ of uniformly random length-n inputs x, then we can construct a randomized algorithm $\mathcal{A}'$ that uses $\mathcal{A}$ as a subrouting to correctly evaluate $f(x)$ for all inputs x with high probability.*

Intuitively, if an efficient algorithm exists for a non-negligible fraction of inputs, then it is easy for all inputs. We are interested in the contrapositive: if the function f is *worst-case hard* (i.e., no polynomial-time algorithm solves it for every input), then it must also be *average-case hard*.

The set of weak instances S_{weak} consists of all x such that $f(x)$ can be computed faster than the average case. If f is not poly-rsr, an attacker could identify and exploit a small subset of weak keys to break the system. Having a poly-rsr f ensures that the difficulty of solving a randomly chosen instance x is on average equivalent to the difficulty of the hardest instance.
This, together with its commutativity ($g^a \cdot g^b = g^b \cdot g^a$), is perhaps behind the widespread interest in DLOG [3].

4.3 Computational comlexity & polynomial hierarchy

We can classify decision problems in complexity classes:

- P, all decision problems D that can be solved by a deterministic algorithm in poly-time.
- NP, all decision problems D that can be solved by a non-deterministic algorithm in poly-time.
- coNP, all decision problems D whose complement $\overline{\mathsf{D}} = \{x \mid x \notin \mathsf{D}\}$ is in NP.

It is known that $\mathsf{P} \subseteq \mathsf{NP}$ and $\mathsf{P} \subseteq \mathsf{coNP}$. The question of whether $\mathsf{P} = \mathsf{NP}$ remains the central open problem in complexity theory.

More generally, for any complexity class C, a problem D is C-*hard* if every problem in C can be reduced to D in polynomial time, and if it also lies in C we call it C-*hard*. This allows to construct the *Polynomial Hierarchy* (PH), a chain of complexity classes Σ_i^P and Π_i^P defined as

$$\begin{aligned}
\Sigma_0^P &= \mathsf{P} & \Pi_0^P &= \mathsf{P} & \Delta_0^P &= \mathsf{P} \\
\Sigma_1^P &= \mathsf{NP} & \Pi_1^P &= \mathsf{coNP} & \Delta_1^P &= \mathsf{P}^{\mathsf{NP}} \\
\Sigma_{i+1}^P &= \mathsf{NP}^{\Sigma_i^P} & \Pi_{i+1}^P &= \mathsf{coNP}^{\Sigma_i^P} & \Delta_{i+1}^P &= \mathsf{P}^{\Sigma_i^P}
\end{aligned}$$

Here notation such as $\mathsf{NP}^{\Sigma_i^P}$ represents oracle access to Σ_i^P. The entire hierarchy is defined as the union of all levels:

$$\mathsf{PH} = \bigcup_{i \geq 0} \Sigma_i^P = \bigcup_{i \geq 0} \Pi_i^P$$

It is known that each level is contained in the next, $\Sigma_i^P \subseteq \Sigma_{i+1}^P$. If there exists an integer i such that $\Sigma_i^P = \Sigma_{i+1}^P$, the Polynomial Hierarchy is said to collapse at the i-th level, implying $\mathrm{PH} = \Sigma_i^P$. It is conjectured that PH does not collapse at any level.

$$\begin{tikzcd}
 & \mathsf{coNP} = \Pi_1^{\mathrm{P}} \arrow[rr] \arrow[dr] & & \Pi_2^{\mathrm{P}} \arrow[rr] \arrow[dr] & & \Pi_3^{\mathrm{P}} & \cdots \\
\Delta_0^{\mathrm{P}} = \Sigma_0^{\mathrm{P}} = \mathsf{P} = \Pi_0^{\mathrm{P}} = \Delta_1^{\mathrm{P}} \arrow[ur] \arrow[dr] & & \mathsf{P}^{\mathsf{NP}} = \Delta_2^{\mathrm{P}} \arrow[ur] \arrow[dr] & & \Delta_3^{\mathrm{P}} \arrow[ur] \arrow[dr] & & \\
 & \mathsf{NP} = \Sigma_1^{\mathrm{P}} \arrow[rr] \arrow[ur] & & \Sigma_2^{\mathrm{P}} \arrow[rr] \arrow[ur] & & \Sigma_3^{\mathrm{P}} & \cdots
\end{tikzcd}$$

The primary utility of poly-rsr functions f in cryptography lies in their guarantees: worst-case hardness implies average-case hardness. Intuitively, if we manage to find an algorithm $\mathcal{A}$ evaluating $f(x)$ on a non-negligible fraction of all possible inputs x, we can construct a randomised algorithm evaluating $f(x)$ for all x. This means that on average we can treat all instances as equally strong (or weak!), so the security of the system relies on the hardest instances, and security proofs can focus on worst-case hardness. However, there is a catch.

► **Theorem 16 ([1])** *If an NP-complete function f is adaptively $O(\log n)$-rsr, then the polynomial hierarchy collapses at the third level.*

Most conjecture that NP-complete problems cannot be poly-rsr, as this would imply a significant collapse in the polynomial hierarchy. Problems such as DLOG lie in $\mathsf{NP} \cap \mathsf{coNP}$ and are believed to not be NP-complete. This also holds for the other problems we saw in Sections 2.1, 2.2, 3.1 and 3.2

Sources and research material

This work is based on the following publications:

[1] *Nonadaptive and adaptive rsr.*
Joan Feigenbaum and Lance Fortnow. "Random-self-reducibility of complete sets". In: *SIAM Journal on Computing* 22.5 (1993), pp. 994–1005.

[2] *In-depth analysis of adaptive k-rsr functions. For an adaptive k-rsr which is not nonadaptive k-rsr, see Section 3.*
Joan Feigenbaum, Lance Fortnow, Carsten Lundj, and Daniel Spielman. "The Power of Adaptiveness and Additional Queries in Random-Self-Reduct ions". In: *IEEE* (1992).

[3] *A compendium of properties of the discrete logarithm problem.*
Antoine Joux, Andrew Odlyzko, and Cécile Pierrot. "The past, evolving present, and future of the discrete logarithm". In: *Open Problems in Mathematics and Computational Science*. Springer, 2014, pp. 5–36. URL: `https://www-users.cse.umn.edu/~odlyzko/doc/discretelogs2014.pdf`.

[4] *See Chapter 3 for isogenies and endomorphisms.*
Joseph H Silverman. *The arithmetic of elliptic curves*. Vol. 106. Springer, 2009.

[5] *Theory and applications of isogenies to cryptography.*
Damien Robert. *On the efficient representation of isogenies (a survey)*. Cryptology ePrint Archive, Paper 2024/1071. 2024. URL: `https://eprint.iacr.org/2024/1071`.

[6] *See Theorem 11 for the result about stationary distributions in the supersingular graph.*
Andrea Basso, Giulio Codogni, Deirdre Connolly, Luca De Feo, Tako Boris Fouotsa, Guido Maria Lido, Travis Morrison, Lorenz Panny, Sikhar Patranabis, and Benjamin Wesolowski. *Supersingular Curves You Can Trust*. Cryptology ePrint Archive, Paper 2022/1469. 2022. URL: `https://eprint.iacr.org/2022/1469`.

[7] *The theory of rsr problems applied to group actions. The author claims an error in the proof of Theorem 15, point 3.*
Giuseppe D'Alconzo. "A note on the hardness of problems from cryptographic group actions". In: *arXiv preprint arXiv:2202.13810* (2022).

[8] *The post-quantum signature scheme LESS, a NIST candidate whose security assumption is based on the hardness of a poly-rsr problem.*
The LESS team. *LESS 2.0*. Tech. rep. 2025. URL: `https://www.less-project.com/resources/`.

[9] *See Section 2.3 for an analysis of rsr for classical problems.*
Dana Angluin and David Lichtenstein. "Provable security of cryptosysterns: A survey". In: *Yale Univ., New Haven, CT, USA, Tech. Rep. TR-288* (1983).

[10] *The first approach to instance-hiding schemes.*
Martin Abadi, Joan Feigenbaum, and Joe Kilian. "On hiding information from an oracle". In: *Proceedings of the nineteenth annual ACM symposium on Theory of computing*. 1987, pp. 195–203.

[11] *Self-correcting schemes.*
Manuel Blum, Michael Luby, and Ronitt Rubinfeld. "Self-testing/correcting with applications to numerical problems". In: *Proceedings of the twenty-second annual ACM symposium on Theory of computing*. 1990, pp. 73–83.